

从SKILLS到LLM-Driven OOP：面向基于LLM的编程语言设计与实现

mp.weixin.qq.com/s/cRJvoqJ4WaQpiFPL_ipA3Q

摘要：随着大规模语言模型（LLM）的兴起，传统的基于指令集架构（ISA）的过程化编程范式在处理海量异构接口与模糊业务意图时，面临着开发困境。本文提出了一种全新的计算架构——**抽象语义计算机 (Abstract Semantic Computer, ASC)**。该架构突破了传统冯·诺依曼体系的物理束缚，将 LLM 抽象为**语义逻辑处理单元 (Language Processing Unit, LPU)**。不同于传统 CPU，LPU 放弃了预定义的二进制指令集，转而直接通过自然语言流进行逻辑推理与指令执行。为了支撑这一硬件变革，本文讨论并实现了一套**原生语义操作系统内核来实现基于语义的寻址**。并且讨论了基于此系统的编程语言 LLM-Driven OOP 并实现了相应的解释器，LLM-Driven OOP 不再规定执行的具体步骤，而是通过定义并由解释器在实行过程中动态组合过程，从而合成出即时生成的执行路径。本文的研究论证了从“人工定义过程”向“内核合成逻辑”转型的可行性，为构建下一代智能化数字化基础设施提供了理论框架与实践路径。

因为恰好SKILLS的爆火，而我们发现我们的工作和SKILLS的思路存在很大的相似性，我们猜测SKILLS的互相调用就是我们的LLM-Driven OOP

1 引言

智能体（AI Agents）的工程实践在过去的两年中经过了两代的演进。第一代智能体（Gen 1）主要依赖 ReAct 循环，将控制权完全让渡给大语言模型（LLM），允许其自主调用有限的外部工具。这种模式虽然灵活，但因缺乏约束导致了不可控的幻觉与死循环。

为了解决可控性问题，第二代智能体（Gen 2）引入了“工作流”（Workflows）概念，试图通过预定义的有向无环图（DAG）或状态机来约束模型的行为。

然而，这种从“完全失控”到“过度控制”的矫枉过正，引发了新的危机：组合复杂度的阶乘级爆炸。随着业务链条的延长，工作流的长度线性增长，但由于每一步都可能存在多种分支与异常状态，整个系统的状态空间实际上呈 $O(N!)$ 级数增长。这种高度特化的“长流程”导致代码复用性极低——一旦业务上下文发生微小变更，整条工作流便需重构。当系统复杂度上升时，这种基于静态编排的工程模式迅速逼近了人类认知的维护边界。

这种“复杂度墙”的出现并非偶然，它揭示了当前主流编程范式中隐含的一种“宏观 OOP，微观过程式”的二元困境。

在架构设计（宏观）层面，开发者利用面向对象（OOP）将代码组织为类与对象，实现了一种空间上的模块化抽象。然而，一旦深入到方法内部的执行（微观）层面，逻辑的本质依然坍缩为严格的过程式结构。无论使用 Java、Python 还是 C++，其执行的基本单元始终是被 CPU 的指令集架构所锁定的线性序列（Linear Instruction Sequences），即循环、if-else 分支与顺序步骤。

这种二元对立导致了难以调和的“流程粒度悖论”（Process Granularity Paradox）：

业务现实是多维的网状结构，达成同一个目标存在多种潜在路径，且最优路径高度依赖于实时上下文。

代码实现却是低维的线状结构，开发者被迫在编码阶段就将所有可能的动态路径“硬编码”为一条条静态的指令序列。

这种不匹配导致开发者必须编写海量的防御性代码来穷举边缘情况，使得智能体开发陷入了“为了确定性而牺牲智能性”的泥潭。

追根溯源，上述工程困境的物理本质在于图灵机的理论限制。经典图灵机模型建立在“读写头在纸带上顺序移动”的假设之上，这决定了现代计算机体系结构在本质上是顺序执行的。虽然多线程技术提供了并发的假象，但核心的逻辑控制流依然遵循确定性的因果链条。

然而，Agent 所面临的现实世界问题求解过程，本质上是一个概率性的、高维的意图搜索过程，而非确定性的指令执行过程。当我们试图用基于“顺序执行”假设的图灵机语言（Python/Java），去模拟基于“概率关联”的大模型推理过程时，不仅产生了巨大的阻抗失配，更在理论层面遭遇了表达力的瓶颈。

面对 $O(N!)$ 的复杂度墙与图灵机顺序执行的限制，近年来，涌现了大量旨在弥合“意图-执行”鸿沟的中间件与工程范式。这些尝试大体可以归纳为两条主要路径。

（1）路径 A：可视化编排与低代码平台

为了应对长流程代码的不可维护性，Dify、LangFlow 等平台通过“可视化编排”将代码逻辑图形化。开发者通过拖拽节点和连线来构建有向无环图（DAG）。这种方法虽然降低了入门门槛，但本质上只是将“代码的意大利面条”转化为了“视觉的意大利面条”。它并没有改变“预定义所有路径”的底层逻辑，只是改变了逻辑的呈现形式。当业务复杂度超过一定阈值，庞大的节点图谱带来的认知负荷甚至远超传统代码，且失去了版本控制与模块化复用的优势。

（2）路径 B：工具调用与 ReAct

为了摆脱编排的繁琐，另一方向（如 Anthropic 的 Model Context Protocol, OpenAI 的 Operator, 以及各类 AutoGPT 变体）转向了“自主”。它们通过赋予模型“工具使用（Tool Use/Skills）”的能力，允许模型在运行时自主规划执行路径。虽然它打破了静态控制流的束缚，却引入了严重的“概率漂移”

。在缺乏明确符号结构约束的情况下，纯粹依赖 LLM 的自我迭代极易陷入死循环或产生幻觉。更关键的是，这种“黑盒执行”在企业级应用中是不可接受的——它无法提供业务逻辑所需的可解释性与边界安全性。它将“编程”退化为了“掷骰子”。

因此我们发现当前的 Agent 开发陷入了一个看似不可调和的“控制-智能困境”：

若追求可控性，我们必须写死流程，牺牲了模型的智能推理能力，回退到传统软件工程的死胡同。

若追求灵活性，我们完全依赖模型推理，牺牲了系统的可靠性，使软件失去了工程严谨性。

现有的解决方案都在这两个极端之间摇摆，却未能找到一种架构，能够同时拥有**OOP 的结构化约束**（确保可靠）与**LLM 的动态推理**（确保灵活）。这正是本文提出 LLM-Driven OOP 的核心动机：我们需要一种能够打破这种二元对立的全新“中间层”。

2 以 LLM 为CPU的语义计算机

正如引言所述，我们认为，当前工业界试图在应用层通过工作流编排或提示词工程不能完全地解决问题。这些方案仅仅是在旧有的计算地基上修补裂缝，而未能触及问题的核心。

为了真正打破“控制-智能”的二元悖论，我们必须从表象层下沉至根源层，即重新审视底层的计算模型。如果我们承认 LLM 已经实质上改变了信息的处理方式，那么我们就不应再将其视为传统程序调用的一个“外部函数”，而应将其视为一种全新的核心处理器。

为此，本文提出构建一台运行在物理机器之上的“抽象语义计算机”（Abstract Semantic Computer, ASC）。这台计算机的设计目标，是打破经典冯·诺依曼架构（Von Neumann Architecture）对“确定性”的依赖，建立一套原生支持“概率性”与“语义性”的计算基质。

在这台虚拟机器中，相应的结构具有不同的含义：

CPU 的演变：从执行确定性算术指令的硅基 CPU，转变为执行概率性语义推理的 LLM-as-CPU。

指针的演变：从指向物理内存地址的“硬指针”，转变为指向业务意图的“语义指针”（Semantic Pointer）。

执行流的演变：从预定义的指令序列，转变为运行时动态生成的“推理链”。

这标志着我们从经典的“冯·诺依曼架构”向“神经符号架构”的基质迁移。

2.1 LPU (Language Processing Unit) : LLM作为CPU

2.1.1 从物理实现到功能性定义的范式转换

传统计算架构建立在确定性的状态转移之上，依赖于物理晶体管开关构建的符号逻辑。然而，在处理企业级复杂意图时，由于人类语言的天然歧义性与业务逻辑的指数级特征，传统基于符号逻辑的 CPU 陷入了“逻辑膨胀（Logic Inflation）”的困境——即试图用穷举的确定性条件分支（If-Else）去模拟弥散的、概率性的现实需求，导致系统复杂度呈几何级数增长。

我们认为，评价一个组件是否具备“处理器”地位，不应受限于其物理实现（如是否由硅基门电路构成），而应取决于其在系统架构中的功能性定义。如果一个单元能够解析复杂指令、管理状态存储、控制外部设备并完成逻辑闭环，那么无论其底层是晶体管开关还是神经网络权重，它在系统拓扑结构中扮演的角色就是CPU。

LPU 的运行逻辑在底层与传统 CPU 具有高度的同构性，但工作在更高的抽象维度：

（1）指令解码：LPU 将模糊的自然语言请求，通过其数千亿参数的权重网络，解码为具体的、可执行的步骤（Plan）。这一过程与传统 CPU 硬件译码器的逻辑高度一致——将高层信号转换为底层可执行的操作序列。

(2) 执行流与程序计数：在 LPU 中，Next Token Prediction（下一个 Token 预测）本质上是程序计数器（PC）的步进。每生成一个 Token，代表指令流向前推进了一个最小计算单位，驱动系统状态从当前位置移向下一个逻辑节点。

基于此，我们将大语言模型（LLM）重新定义为LPU (Language Processing Unit)。在学术界，这一趋势已得到广泛认可，如 Andrej Karpathy (2023) 提出的“LLM OS”构想，以及AIOS (LLM-based Operating System, 2024) 等前沿研究，均证实了将神经网络作为系统内核处理器的可行性。

2.1.2 计算本质：从确定性数学到概率性语义推理

LPU 的计算模式不同于传统 CPU 遵循的布尔逻辑，LPU 运行在纯粹的概率计算之上。LPU 的操作被定义为从“语义”到“语义”的映射。给定一个输入语义空间，LPU 通过其内部数十亿个参数构成的概率分布，计算并输出另一个对应的语义空间。

正如 Andrej Karpathy 所指出的，我们应当区分两种本质不同的“处理器”能力

传统 CPU：擅长确定性的数学计算。在处理 1+1=2 或精确的内存寻址时，它拥有绝对的可靠性，但在处理模糊的自然语言意图时会遭遇“逻辑膨胀”。

LLM 这种“CPU”（LPU）：擅长不确定的语义推理。它处理的是模式识别、逻辑类比与意图解析，这些任务在传统 CPU 上极难被硬编码（Hard-coded），但在概率模型中却是原生支持的。在 ASC 体系结构中，LPU 不负责具体的加减乘除，而是负责“理解”与“规划”。它将复杂、模糊的外部需求，通过概率推理转化为一系列可供外设执行的逻辑步骤。这种架构承认了世界的不确定性，并利用概率计算来消除由于语言歧义带来的执行障碍。

2.2 存储分级与映射：从 RAM 到语境窗口

为了实现高效的语义计算，ASC 建立了一套与传统计算机对标的存储层次结构。

传统组件	ASC 组件	功能定义
RAM (内存)	Context Window	存储当前进程的即时状态。基于注意力机制，窗口内数据可被 LPU 即时寻址。
Disk (二级存储)	Vector DB / RAG	作为持久化的外部存储，解决 LPU 权重参数量有限的问题，实现大规模知识挂载。
MMIO (内存映射)	Unified State Space	将异构数据库（SQL/Neo4j）抽象为 LPU 的物理寻址空间，实现“按需寻址”。

传统的计算机内存管理致力于在寄存器、RAM 和磁盘之间建立高效的数据调度机制。我们定义了一套同构的语义存储层次结构（Semantic Memory Hierarchy），解决了 LLM 有限上下文窗口与海量异构数据之间的矛盾。

(1) 统一地址空间 (Unified Address Space)

尽管物理数据分散存储在本地 SQL、向量数据库或第三方 SaaS（如 Salesforce, ERP）中，但在 AVM 的视角下，它们被扁平化为一个“统一语义状态空间” (Unified Semantic State, USS)。

异构数据的同构抽象：NS-VM 打破了“内存数据”与“I/O 数据”的传统界限。无论是访问本地数据库的一行记录，还是通过 API 获取 SaaS 中的订单详情，在虚拟机层面均被视为标准的“内存读取” (Memory Load) 操作。

虚拟化层 (Virtualization Layer): AVM 内部通过第2章所述的上向流模型将语义指针解析为具体的物理获取方式

(2) 上下文分页机制 (Context Paging Mechanism)

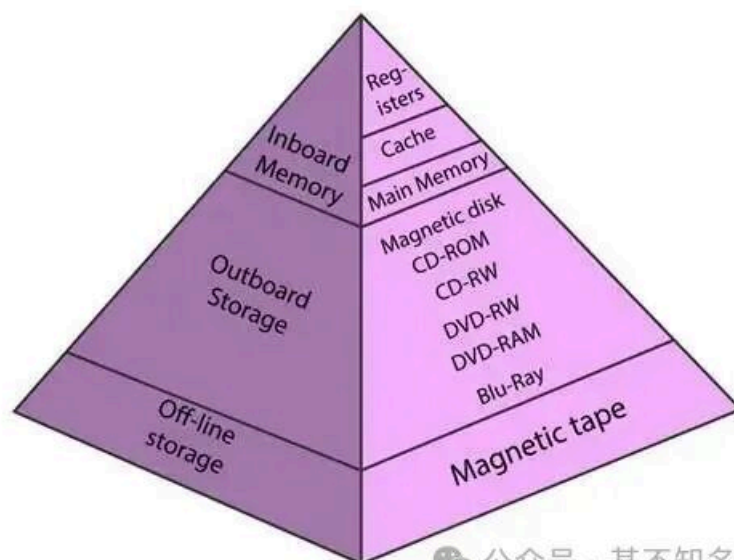
这是内存管理的核心。LLM 的 Context Window（上下文窗口）极其昂贵且有限，相当于传统计算机的 L1 缓存/寄存器；而 SaaS/DB 是无限但缓慢的主存/磁盘。

因此需要“按需换入” (Just-in-Time Hydration) 策略：

缺页中断 (Page Fault): 当 Agent 需要思考时，它持有的只是指向数据的语义指针（轻量级引用）。如果 LPU（LLM）发现当前 Context 中缺少该指针的具体内容，会触发“语义缺页中断”。

数据换入 (Swap-in): AVM 拦截该中断，根据指针的定义（是 SQL 还是 SaaS API），自动执行查询操作，将返回的 JSON/Text 数据加载 (Load) 到 LLM 的当前 Context 中。

脏页写回 (Dirty Write-back): 当 LPU 完成推理并修改了对象状态后，AVM 并不会只修改 Context 里的文本，而是生成对应的 UPDATE SQL 或 POST API 请求，将变更同步回物理存储 (SaaS/DB)。



塔尖 (L1 Cache): LLM Context Window (容量小，速度快，负责计算)。

塔身 (Main Memory): Hot Data in Vector DB/Redis (近期记忆)。

塔底 (Disk/Storage): SaaS APIs & SQL Database (全量数据，持久化)。

2.3 语义总线：I/O 设备与异构协作

ASC 通过语义总线将外部世界接入 LPU，实现了对物理和数字工具的统一调度。

2.3.1 I/O 设备：工具与 API 的外设化

在 ASC 架构中，外部工具不再是孤立的软件，而被视为计算机的 I/O 设备：

计算外设：如 Code Interpreter、Calculator。

感知/行动外设：如 Browser（浏览器）、各类 SaaS API。

交互协议：LPU 通过 Function Calling（函数调用）机制实现对这些外设的控制。这相当于传统计算机中的总线中断或寄存器写入，将 LPU 的语义意图转化为物理世界的具体操作。并且外部系统返回原始数据（如 JSON、XML 或 SQL Result Set）。执行引擎负责将这些数据标准化为统一的语义对象，并回传给上层的解释器，作为当前步骤的执行结果。

3 语义计算机的系统与内存管理

基于前述的抽象语义计算机（ASC）架构，我们需要构建系统的**内核（Kernel）**，接管 ASC 的硬件资源，为上层智能体代码提供了一个统一的、具备自动内存管理与即时编译能力的执行环境。

3.1 下行流：基于语义调用栈的递归分解

LPU 本身是**无状态**的。AS-Kernel 的核心组件之一是‘虚拟栈管理器’。它不再像传统 OS 那样仅仅分配内存，而是直接参与逻辑寻址，将非过程化的拓扑指令转化为具有时序关系的栈操作。AS-Kernel 维护的链表栈，实际上充当了 LPU 的“**外部逻辑寄存器组**”。在一个分布式数据库中，“下一行记录”并不在物理地址的“下一个字节”。因此，依赖物理连续性的数组结构在 ASC 中在理论上是不可行的。因此整个是基于链表来实现的。

3.1.1 从指令指针到意图指针

在经典解释执行模型中（如 CPython），CPU 依据一个明确的指令指针（Instruction Pointer, IP）顺序遍历编译后的操作码。分支逻辑（如 JMP、CALL）是数学上确定性的。

而在 ASC-OS 中，我们引入了意图指针（Intent Pointer, ）。执行流不再由内存地址偏移驱动，而是由语义相似度引导。

传统调用：Result = ObjectA.methodB(Args)（硬编码依赖）；

ASC-OS 调用：Result = Context.Resolve(Goal)（软性、概率性依赖）。

直到执行发生的那一刻，才确定下一个将被调用的智能体。这体现了极致晚绑定——不仅推迟到运行时，更推迟到语义推理的瞬时时刻。

2.3.1.2 分发机制：语义虚函数表

为支持动态组合，我们将传统的虚函数表（v-table）替换为语义能力声明。每个智能体类暴露一个能力（以自然语言描述其功能边界）。

分发循环的工作流程如下：

- (1) 意图感知：当前栈帧持有一个高层目标 G （例如：“提升用户留存率”）；
- (2) 候选评估：运行时扫描可用子智能体注册表
- (3) 概率匹配：LLM 计算语义概率—即智能体 S_i 有效解决目标 G 的可能性；
- (4) 动态链接：解释器选择亲和度最高的智能体们：；
- (5) 实例化：创建新实例，并将其压入栈。

关键在于，这种“链接”是瞬态的。如果上下文发生变化（例如用户身份从“新用户”变为“VIP”），解释器在下一次运行中可能会选择完全不同的智能体。因此，逻辑路径从未固化；它是流动的、情境敏感的。

3.2 上行流：基于语义引用的垂直上下文搜索

在ASC-OS 架构中，内存管理远不止于存储分配；它是一座语义桥梁，负责将非结构化意图（由高层智能体持有）转化为结构化参数（为低层工具所必需）。我们提出了一种新型的内存模型，该模型在逻辑上是对传统操作系统虚拟内存机制的语义形式的重构。

我们将这一机制定义为“倒置的引用解析”（Inverted Reference Resolution）。它将传统的“值传递”转变为“语义引用传递”，并将变量的赋值过程推迟到执行的最后一刻（Lazy Materialization）。ASC-OS 内存模型是一个构建在数据库之上的虚拟内存系统，通过自然语言来寻址。

2.3.2.1 符号表：基于 Schema 强制的数据作用域

与传统解释器中变量类型由编译器预先定义不同，上层语言通过 DataScope Schema 来声明变量。每个栈帧

在实例化时都会创建一个本地符号表（Symbol Table），该表严格受限于其所属的 Schema。

作用：使LLM 可能以自然语言“思考”（例如：“用户想要红色鞋子”），但符号表只接受符合类型约束的结构化数据（例如：`product_id: "sku_123", color: "red"`）。

结构形式化如下：

其中：

k 为变量名，

v 为实际值，

t 为 Schema 中定义的类型约束。

2.3.2.2 垂直上下文搜索算法

在传统的静态语言中，若函数调用参数必须显式传递。然而在ASC-OS 的动态协作链中，顶层智能体（战略层）往往只持有模糊的业务意图，无法预知底层工具（执行层）所需的精确参数签名。

为了解决这一“参数不可知悖论”，我们设计了垂直上下文搜索算法。该算法将变量绑定过程分为两个阶段：

阶段一：语义锚定与指针合成

(1) 初始态：符号

当底层工具被调用时，解释器面对的是一个裸露的变量名（如 `client_id`）。此时，该变量仅仅是一个待填充的符号，没有任何指向性。

(2) 第一级补完：局部意图绑定

系统首先扫描当前栈帧的局部上下文。

机制：解释器分析当前动作的直接指令。

示例：当前指令是“核查该用户的退款资格”。

结果：变量 `client_id` 获得了第一层语义附着：它不再是任意 ID，而是“当前正在处理退款的那个用户的 ID”。但“该用户”的具体身份在局部依然是指代不明的。

(3) 第二级补完：记忆重构

为了消除局部上下文中的代词歧义（如“该用户”、“他”），系统沿调用链（Call Chain）向上回溯，检索父级智能体的业务记忆。

机制：解释器将局部意图与父级对话历史进行语义对齐。

示例：父级智能体（战略层）在上一轮对话中记录了：“任务目标：处理昨天的第二个客户的投诉”。

合成：系统将父级的精确描述（“昨天的第二个客户”）注入到子级的局部指代（“该用户”）中。

经过两级补完，原本空洞的变量名 `client_id` 被转化为一个完整的、可执行的语义指针：

Pointer: `client_id` ↪ "The ID of the second customer from yesterday who requires a refund qualification check."

这个“指针”现在是一个自包含的语义实体，它作为“语义指针”传递给下一阶段的所有权路由机制。它包含了足够的信息量，使得无论数据归属于哪个类，该类都能仅凭此描述生成精确的 SQL 查询，而无需再次回调上层询问细节。

阶段二：语义寻址

在传统 OS 中，CPU 拿着虚拟地址去查页表（一级、二级页表），找到对应的物理页框号。而在ASC-OS，地址是基于语义的。

当控制流抵达底层叶节点（例如 API 调用层），且检测到方法签名需要一个具体的结构化参数（如 `int client_id`）时，系统触发**即时实体化**。通过逐级检索 DataScope Schema，将“语义虚拟指针”翻译为“物理数据库地址”的过程。该过程包含三个寻址阶段：

(1) 语义页表遍历：逐级向上寻址

机制：在 x86 架构中，逻辑地址由段选择符:偏移量（Segment:Offset）组成。CPU 首先根据段选择符在全局描述符表（GDT）中找到段基址（Segment Base）。在 LLM-Driven OOP 中运行时持有待解析的变量名（如 `client_id`）和当前上下文中的语义指针（如“昨天的第二个客户”）。系统并不直接去数据库查，而是沿着执行栈（Stack）和继承链（Inheritance Chain）逐级向上遍历。

匹配逻辑：在每一层级，解释器检查当前类的 DataScope Schema。Schema 充当了“页表项”的角色，声明了该类管辖的数据字段。

命中：系统在某一层级（例如 CustomerManager 类）的 Schema 中找到了 `client_id` 的定义。这标志着寻址成功：我们找到了该数据的归属类（Owner Class），即负责解释该语义指针的逻辑实体。

(2) 地址翻译与指令合成

机制：一旦锁定归属类，系统将“语义指针”（自然语言描述）移交给该类的解释器。

转换：归属类充当**动态翻译器**——它利用其内部持有的领域知识（Domain Knowledge）和数据库 Schema，并其内部定义的转换逻辑（Text-to-SQL 实现）将模糊的自然语言转化为精确的数据查询指令。

输入（语义指针）： "The second client from yesterday"

输出（物理地址指令）： `SELECT id FROM clients WHERE created_at = DATE('now', '-1 day') LIMIT 1 OFFSET 1`

这一步实现了从“语义空间”到“物理查询空间”的映射。

(3) 物理加载

实体化：数据库返回物理值 8821。该值被回填至底层的参数槽位中，完成变量的查询。

这种“引用下传，寻址上溯”（Reference Down, Addressing Up）的机制，确保了数据的单一事实来源（Single Source of Truth）。变量看似是在调用链中传递，实则传递的只是一个轻量级的语义指针。只有当 DataScope Schema 确认了归属权后，指针才会被解析为昂贵的数据库 I/O 操作。

4 适配于语义系统的 LLM-Driven OOP

正如汇编语言是为了适配物理 CPU 的寄存器操作，Java 是为了适配 JVM 的字节码操作，新的计算基质必然呼唤新的编程语言。现有的 Python 或 Java 是为确定性的物理图灵机设计的，它们缺乏原生的语法结构来描述“语义指针”的跳转或“概率意图”的坍缩。因此，我们需要一种新的

语言抽象层——即本文提出的 LLM-Driven OOP。它是这台“认知虚拟机”的指令集，专门用于定义那些能够被 LLM 理解并动态执行的“模糊对象”与“语义方法”。

在确立语言的核心类型系统时，我们并未沿用 Java 或 C++ 等传统面向对象语言中基于“继承（Inheritance）”的深层层级结构，而是深刻借鉴了 Go 语言（Golang）的设计哲学——即“组合优于继承”（Composition over Inheritance）与“结构化类型”（Structural Typing）。

（1）传统 OOP 试图通过复杂的继承树（Inheritance Tree）来静态地模拟世界（例如：AdminUser 继承自 User）。然而，我们认为，“人的业务逻辑是组合式的”——一个复杂的任务通常由若干个原子能力的灵活拼装而成。因此，我们采用了类似 Go 语言的“正交组合”范式：因此，我们采用了类似 Go 语言的“正交组合”（Orthogonal Composition）范式。在 LLM-Driven OOP 中：

没有“类”的层级：对象不再被视为分类树上的一个节点，而是被视为一组能力的容器（类似于 Go 的 struct）。

能力即接口（Skills as Interfaces）：我们将业务能力封装为原子化的 Skill（对应 Go 的 interface）。

动态装配：一个智能体（Agent）的定义，本质上就是一系列 Skill 的嵌入（Embedding）。

这种设计使得对象结构变得极度扁平化（Flattened）。它允许 LPU（LLM）在运行时根据上下文，灵活地调用附着在对象上的任意能力，而无需关心对象在继承树中的位置。这不仅降低了编译时的耦合度，更重要的是，它为 LLM 提供了最大化的推理自由度（Reasoning Freedom）——模型可以像搭积木一样，根据意图动态组合不同的 Skill 来解决问题。

在这种组合式架构下，代码的角色发生了转变。代码不再描述“控制流是如何一步步执行的”（这是 LPU 的工作），而是描述“哪些能力是被允许组合的”。开发者通过定义 Schema 和 Skill 接口，构建了业务逻辑的“骨架”与“边界”；而 LLM 则在运行时填充动态的“血肉”。

4.1 LLM-Driven 语言规范：分形智能体模式

在传统面向对象编程（OOP）中，“类”（名词）与“方法”（动词）之间存在严格的二元对立。LLM-DRIVEN OOP 打破了这一界限。我们提出分形智能体模式（Fractal Agent Schema），其中每一个处理单元——无论是高层的领域控制器，还是底层的原子动作——在结构上都是同构的。这种自相似性（self-similarity）赋予系统无限的可组合能力。

4.2 语法表示

尽管运行时内部以图结构进行操作，但源代码采用人类可读的 YAML 格式编写。以下是一个私域营销智能体（Private Domain Agent）的示例：

Code 1：示例

标识与拓扑约束

Class: PrivateDomainMarketing

ID: private_domain

“语义虚函数表”：用于任务分发

#描述该智能体负责处理的业务意图

Capability: >

[核心] 构建“用户分群—裂变机制—活动策划—执行落地”的闭环。

[动作] 用户分群、裂变机制设计、活动方案制定。

[目标] 通过企业微信与小程序管理私域流量。

#本地内存栈帧（数据作用域）

#本层级严格管理的变量

DataScope:

-user_unique_id # 类型: String

-wechat_work_id # 类型: String

-community_id # 类型: Integer

-interaction_tags # 类型: Set

-conversion_status # 类型: Enum(Unreached, Intention, Deal)

-source_channel # 类型: String

实现层（递归子智能体）SubAgents:

-AgentRef: fission_mech_design 1

-AgentRef: copy_planning 2

-AgentRef: user_strat_fission 3

-AgentRef: rules_system_create_active_center 4

该 YAML 结构直观体现了 LLM-Driven OOP 的核心思想：

- (1) 能力由自然语言声明，而非硬编码逻辑；
- (2) 状态被封装在局部作用域内，避免全局污染；
- (3) 行为通过嵌套子智能体递归组合，形成分形式的执行树。

这种设计使开发者能够以声明式方式构建复杂、可解释且可演化的自主智能体系统。

4.3 语法树构建：全同构语义蓝图

为了适配 LPU（语言处理单元）的认知特性，我们在代码的语法分析（Parsing）阶段实施了激进的结构归一化（Structural Normalization）策略。不同于传统编译器侧重于解析“控制流”，本系统的 AST 旨在构建一张全同构的“语义意图蓝图”。具体而言，我们提出的 AST 并非对经典结构的摒弃，而是一种抽象。经典 AST 将源代码分解为供确定性 CPU 使用的原子操作指令，而我们的系统将源代码解析为供概率性 LLM 使用的粗粒度语义单元（Actor 定义）。这代表了从语法树向组织蓝图的转变——源代码的层级结构隐含了智能体网络的监管拓扑，而非控制流的顺序。

4.3.1 节点的同构化设计 (Node Homogeneity)

AST 的节点形态直接取决于底层计算单元的指令集架构（ISA）：

传统 CPU 的约束：传统 CPU 拥有复杂的异构指令集（如算术指令、跳转指令、访存指令）。为了适配这种架构，经典 AST 必须是高度异构的，需要将代码精确切割为 BinaryExpression、IfStatement、MethodDeclaration 等数十种微粒，以便后续生成对应的汇编指令。

本系统 LPU 的特性：LPU 本质上是一个“单指令计算机”——它唯一的输入接口是上下文窗口（Context Window），唯一的原子操作是文本生成（Text Generation）。对于 LPU 而言，解析一个“类定义”与解析一个“条件判断”并无本质区别，它们都是需要被理解的自然语言上下文。

基于此，为了适配 LPU 的单一接口特性，我们执行了强制性的“分类坍缩”：我们将 AST 中所有原本区分的结构性节点（类、模块）与逻辑性节点（函数、复杂语句块）统一泛化为唯一的节点类型——ActorDefinitionNode。这棵树不再是异构指令的森林，而是适配 LPU 输入格式的同构语义容器网络。

4.3.2 静态蓝图：提示词上下文的封装容器

在 ASC 架构下，AST 节点的角色从“操作码生成器”转变为“LPU 上下文载荷（Context Payload）”的容器。这棵同构树通过预处理，将源代码转化为 LPU 可直接“吞吐”的格式。每个 ActorDefinitionNode 包含以下核心要素，为后续的解释器实例化做准备：

Identity (身份标识)：节点在源码中的唯一路径（用于寻址）。

Gene/Prompt Context (基因/提示词上下文)：这一部分是核心。源代码的文本内容（如函数体内的注释、逻辑描述）被直接封装为该节点的“元系统提示词（System Prompt）”。这确保了当该节点被实例化为 Actor 时，它“生来”就知道自己的职责。

Topology (拓扑连接)：定义了该节点与其子节点（Children）及兄弟节点（Siblings）的静态连接关系。这决定了运行时的消息路由通道。

4.3.3 语义粒度：从微操指令到意图块

这种适配 LPU 的 AST 设计，实现了计算粒度的改变：

CPU 视角(How)：关注执行细节。经典 AST 解析 `for (i=0; i<n; i++)` 是为了生成寄存器操作和跳转指令。

LPU 视角(What)：关注意图整体。本系统 AST 不再解析循环或分支的内部结构。我们将整个 `for` 循环体视为一个不可分割的 ActorDefinitionNode。

理由：LPU 不需要通过 JMP 指令来执行循环，它具备内建的逻辑推理能力。它只需要读取“遍历订单列表并检查状态”这一段完整代码块（Context），就能在思维链中直接推导出结果。我们将控制流的解析权从编译时的语法分析器移交给了运行时的 LPU。

4.3.4 分形架构 (Fractal Architecture)

由于 LPU 对不同长度上下文的处理逻辑是一致的（处理 10 行代码的函数与处理 100 行代码的类，本质都是 Token 预测），这棵同构语法树呈现出完美的**分形特征**：

宏观：整个程序是一个巨大的 Context 容器（根节点）。

微观：模块、类、方法、甚至复杂的 if-else 块，都是嵌套的子级 Context 容器。

这种结构保证了 ASC 系统的**尺度不变性 (Scale Invariance)**。解释器无需针对不同层级切换处理模式，只需递归地将每一个节点的内容加载到 LPU 中即可完成实例化。

5 解释器实现

5.1 基于Actor的LLM-Driven OOP解释器实现

与传统面向对象编程（OOP）中调用图（Call Graph）在编译期即被静态确定不同，LLM-Driven OOP 运行时的决定性特征在于它彻底摆脱了冯·诺依曼执行模型——在该模型中，指令序列由程序员预先确定。相反，LLM-Driven OOP 实现了即时逻辑合成（Just-In-Time, JIT Logic Synthesis），采用一种分形执行模型（Fractal Execution Model）。由于执行路径（控制流）是即时合成的，数据流必须支持极致晚绑定——即变量的最终形态和绑定时机被推迟到语义推理的最后一刻。这是 LLM-Driven OOP 区别于标准 LLM 链式调用的架构核心。传统方法要求上下文作为参数严格向下传递；而 LLM-Driven OOP 中由于动态合成任务的具体方法签名（所需参数）在运行时之前无法预知

本系统的解释器设计彻底摒弃了传统解释器“单体循环 + 遍历执行”的模式。解释器的职责就变成了：按需激活这些节点，让它们根据上下文自主协作。因此，本系统的解释器实现为一个“思考型 Actor”（Thinking Actor）的分布式集群。解释过程本质上是自然语言意图（Intent）在 Actor 网络中的递归分解与分发过程。

系统中的每一个 Actor 都是源码中 ActorDefinitionNode（见上一章）的运行时实例。由于 AST 是同构的，所有的 Actor 在运行时的行为差异，完全取决于“注入的源码上下文”。

一个标准的 ThinkingActor 包含以下核心组件：

Blueprint (蓝图): 指向 AST 中对应的 ActorDefinitionNode，定义了它的职责边界。

Mailbox (邮箱): 异步接收 Msg_Task（任务）和 Msg_Result（子节点反馈）。

State (状态): 持久化在 USS (SQL) 中的当前上下文，包括正在等待的子任务列表。

当一个 Actor 收到父节点发来的任务消息时，它并不直接执行原子操作，而是启动一个“思考-分发”循环。这一过程模拟了人类管理者处理复杂任务的流程：

步骤 I：意图解析 (Intent Parsing via LPU)

Actor 将接收到的Msg_Task（包含自然语言描述的任务）与自身的 Source Code（源代码定义）一并提交给 LPU（大语言模型）。

Prompt 示例：“我是对象 [OrderManager]，我的源代码如下... 现在收到任务‘处理订单’。请根据我的代码逻辑，将此任务分解为一系列具体的子步骤。”

步骤 II：递归分解 (Recursive Decomposition)

LPU 进行推理，将高层意图分解为一组有序或并发的子意图（Sub-Intents）。

例如，任务“处理订单”被分解为：

- 1.向UserActor 确认用户余额。
- 2.向InventoryActor 扣减库存。
- 3.向LogActor 记录日志。

步骤 III：动态分发 (Dynamic Dispatching)

Actor 根据分解结果，执行以下操作：

- 1.寻址：利用语义指针（Semantic Pointer）在 USS 中查找或实例化对应的子 Actor。
- 2.发信：将子意图封装为新的 Msg_Task，异步发送给子 Actor。
- 3.挂起：当前 Actor 并不阻塞等待，而是将自身状态标记为 WAITING_FOR_CHILDREN，并将上下文快照写入 USS，释放计算资源。

步骤 IV：结果聚合 (Result Aggregation)

当子 Actor 完成任务并回复 Msg_Result 时，父 Actor 被重新唤醒。LPU 再次介入，评估所有子任务的结果：

- 如果所有步骤完成：向父级汇报成功。
- 如果中间步骤需要依赖前一步的结果：触发下一轮分发。

在本系统中，传统的“调用栈”（Call Stack）**不再以连续内存块的形式存在，而是被重构为“分布式因果链”（Distributed Causal Chain）**。在传统解释器中，控制流由宿主语言（如 Python）的调用栈（Call Stack）管理。在 Actor 实现中，我们必须显式重建这一机制。

虽然物理上不存在全局栈，但每一条消息都携带了 TraceID 和 ParentActorRef。这些元数据在分布式网络中隐式地串联起一个逻辑调用栈。消息的流转路径（Message Flow）即构成了程序的执行轨迹。

异常冒泡（Exception Bubbling）：

当底层 Actor（终结符节点）执行失败时，它向父 Actor 发送 `Msg_Failure`。父 Actor 的监督策略（Supervision Strategy）捕获该消息，并决定是重试（Retry）还是向上一级继续抛出异常。这精确复刻了解释器模式中的 try-catch 异常处理机制。

5.2 可观测性与软控制实现

为了在非确定性的 LLM 执行环境中确保系统的安全性与稳定性，我们引入了独立的治理层。

该层严格遵循控制平面（Control Plane）与数据平面（Data Plane）分离的原则：数据平面负责业务逻辑的推理与执行，而控制平面通过“全局状态注册表”与“信号总线”，实现了一种创新的“软控制”（Soft Control）机制。这一设计将传统 AI 系统不透明的“黑盒（Black Box）”转化为可视可控的“白盒（Glass Box）”，从而原生支持人在回路（Human-in-the-Loop, HITL）的实时干预与系统级调试。

5.2.1 上行遥测：心跳与动态拓扑映射

为了实时捕捉系统的运行状态，我们设计了一套基于事件驱动的上行遥测机制。

语义心跳：在执行任何工具调用、状态变更或子任务分解之前，Thinking Actor 必须向全局信号总线发布一个轻量级的遥测数据包。该数据包不仅包含存活信号，还携带当前的语义摘要（例如：“正在思考订单拆分逻辑...”）。

实时拓扑重构：全局状态注册表（Global State Registry）作为唯一的消费端，持续聚合这些遥测流。它动态维护着一张分形执行图，精确映射出当前活跃的 Actor 节点及其父子引用关系。

这使得外部观察者（如监控仪表盘或 IDE 调试器）能够以“上帝视角”实时透视整个智能体网络的思维链路，实现了细粒度的可观测性。

5.2.2 下行干预：信号检查门与控制原语

为实现“软控制”，所有智能体被设计为可中断的。我们在前述的执行循环中，强制插入了一个“信号检查”。

检查机制：在 Actor 进入耗时的 LPU 推理阶段之前，必须轮询信号总线。检查逻辑支持层级传播（Hierarchical Propagation）：Actor 不仅检查针对自身的信号，还检查针对其所有祖先节点（Parent/Grandparent）的信号。这意味着，暂停根节点将自动冻结整棵任务树。

控制原语集 (Control Primitives)：我们定义了四种标准的软控制信号，用于精细化干预 Agent 的行为：

`SIG_NOOP`（继续）：正常执行；

`SIG_PAUSE`（暂停）：智能体冻结当前栈帧，进入自旋等待状态，直至收到恢复信号。此功能支持单步调试（Step-by-Step Debugging）；

`SIG_TERMINATE`（终止）：智能体立即中止执行，对所有未提交的数据库事务执行回滚，并从认知栈中弹出自身；

`SIG_INJECT`（注入）：智能体在进入下一步推理前，将外部人工反馈注入其语义上下文，实现动态修正。

与粗暴的“硬控制”（如直接杀死进程）不同，这种“软控制”机制支持优雅降级与动态航向修正，而不会导致运行时崩溃，从而在保持系统鲁棒性的同时，赋予人类对复杂 AI 行为的实时监督与干预能力。

6 评估

暂缺，可直接访问项目github.com/Kaminarusawa-sekai/Flora

7 总结

在本研究中，我们正式提出了 LLM-DRIVEN OOP——一种融合了神经符号主义（Neuro-Symbolic）的新型计算范式。这项工作旨在解决复杂业务与有限条件之间的矛盾。

早在数十年前，艾伦·凯（Alan Kay）等先驱便曾构想：面向对象编程不应仅是用于管理数据结构的工程工具（如 C++ 或 Java 中的实践），而应模拟一个**生物系统**——其中，自主的“细胞”通过消息传递，以适应性方式协同解决复杂问题。然而，这一愿景长期停留在理想层面，受限于当时缺乏具备**语义推理能力**的计算底座。过去的“对象”虽在架构上合乎规范，却在认知上完全惰性。

大语言模型的出现，终于为我们补全了缺失的核心组件——即**像处理数据一样原生地处理意图**（Intent）的能力。

LLM-DRIVEN OOP 不仅仅提出了一种新语法；它**复活并真正实现了 OOP 先驱们最初未竟的梦想**。我们不再仅仅模拟生物细胞的结构，而是开始模拟其**能动性**（Agency）——那种感知环境、理解目标、自主决策并协同行动的生命特质。